

Examples of Structuring Patterns

Conversation Log | June 27, 2026

1. Decomposition Pattern

USER

give me an example of the Decomposition pattern

CLAUDE

Here's a concrete example of Decomposition in action — using something close to your own work, building a TextFinder feature in Rust.

Vague, non-decomposed prompt:

“Build a text search tool in Rust that finds patterns in directory trees and shows match context.”

This asks the model to solve file traversal, pattern matching, output formatting, and CLI parsing all in one undifferentiated blob — easy for it to drop a requirement or tangle concerns together.

Decomposed version:

“I want to build a recursive text-finder in Rust. Let's break it into steps and tackle them one at a time:

1. Define the CLI argument structure (pattern, directory, file extensions, case sensitivity)
2. Implement directory traversal that yields candidate file paths
3. Implement line-by-line pattern matching within a single file, returning line number + matched line
4. Wire traversal and matching together, with one file's worth of results at a time
5. Format and print results (grouped by file, with line numbers)

Start with step 1 only. Show me the CLI struct, then stop so I can review before we move to traversal.”

Why this works better:

- Each step has a clear, checkable output (you can compile/review step 1 before step 2 exists)
- Mistakes are caught early and locally instead of buried in a 200-line dump
- It mirrors how your RustCmdLine / RustDirNav crates are already separated — decomposition here just makes that separation an explicit part of the prompt instead of something you reconstruct after the fact
- It's the same instinct your `sdd:plan` → `sdd:implement` flow already encodes at a larger scale: decomposition is what `tasks.md` is, just zoomed in to a single response instead of a whole feature

The general shape: instead of asking for the whole thing, name the subtasks up front, fix their order, and ask for one at a time (or ask the model to do them in order with checkpoints between).

2. Planning Pattern

USER

give an example of the Planning pattern

CLAUDE

Planning is closely related to Decomposition but distinct: Decomposition splits a task into parts, while Planning is about getting the model to propose a full plan up front and pause for your approval before any execution happens. The emphasis is on the approval gate, not just the breakdown.

Without Planning (jumps straight to execution):

“Add a `--regex` flag to `TextFinder1` so it supports regex patterns instead of just plain substrings.”

The model just starts writing code immediately — picking a regex crate, deciding how it interacts with existing CLI args, deciding error handling — all without you seeing those decisions until the diff is already in front of you.

With Planning:

“I want to add a `--regex` flag to `TextFinder1` so it supports regex patterns instead of just plain substrings. Before writing any code, give me a plan covering:

- Which crate you'd use and why
- How the flag interacts with existing CLI args (mutually exclusive with current matching mode, or layered on top?)
- What changes to `RustCmdLine` vs the matching logic in the core crate
- Error handling if the user passes an invalid regex

Don't write any code yet — just the plan. I'll review and tell you to proceed.”

The model responds with something like:

Plan: I'd use the regex crate (already a near-standard choice, no extra OS deps). I'd add `--regex` as a boolean flag in `RustCmdLine`'s arg struct, mutually exclusive with substring mode — trying to set both would error out at parse time. The core matcher would take an enum (`Substring(String)` or `Regex(Regex)`) instead of a raw string, so the rest of the matching logic doesn't need to branch internally. Invalid regex syntax would be caught at startup via `Regex::new()`'s `Result`, printed as a clean CLI error, and exit before touching the filesystem. Open question: should `--regex` also affect how match context is highlighted in output, or just how matching works?

Then you approve, push back, or answer the open question — before a single line of implementation exists.

Why it matters separately from Decomposition:

a plan can be perfectly decomposed into steps and still be wrong — wrong crate choice, wrong architectural boundary, missed edge case — and you'd only find out after the code's written. Planning catches that earlier by making the model commit to its reasoning in plain language first. This is exactly what your `sdd:plan` command does at the feature level: draft `plan.md`, set its status to Draft, and require your approval before `sdd:implement` is allowed to touch any source file.

3. Spec-Driven Pattern

USER

give an example of the Spec-Driven pattern

CLAUDE

Spec-Driven Development goes a layer beneath Planning: Planning asks “how will you build this?”, but Spec-Driven asks “what exactly are we building, and what counts as done?” — answered and locked in before any planning happens. The spec is the contract; the plan is just one way to satisfy that contract.

Without a spec (plan with no fixed target):

“Add support for searching multiple file extensions at once in TextFinder1, then plan how to build it.”

The model can plan against its own guess of what “multiple extensions” means — comma-separated list? repeated flags? a config file? It might guess differently than you intended, and you won't notice until the plan (or worse, the code) doesn't match what you actually wanted.

With a spec first:

“Before any planning or code, let's write a spec for multi-extension search in TextFinder1. I want it to cover:

- Goal: what problem this solves
- Behavior: exact syntax for specifying extensions, and how it interacts with the existing single-extension flag
- Edge cases: no extension given, malformed extension string, extension with/without leading dot
- Out of scope: anything we're explicitly not doing (e.g. extension wildcards)
- Acceptance criteria: how we'll know it's done correctly

Leave anything unclear as an open question for me instead of guessing. Don't draft a plan yet.”

The model responds with something like:

Goal: Let users search across several file extensions in one run instead of one extension per invocation.

Behavior: New `--ext` flag accepts a comma-separated list, e.g. `--ext rs,toml`. Coexists with current `--ext` single-value usage — single value is just a list of one.

Edge cases: Empty string after `--ext` → error before traversal starts. Extensions given with or without leading dot (`rs` vs `.rs`) → normalize to one internal form.

Out of scope: Glob patterns like `--ext *.rs` — that's a separate feature.

Open question: Should extension matching be case-sensitive (`RS` vs `rs`)? Windows/your filesystem context makes this ambiguous — what do you want?

Acceptance criteria: Given a directory with `.rs`, `.toml`, and `.md` files, `--ext rs,toml` returns matches only from the first two.

You answer the open question, mark the spec Approved — and only then does planning (architecture, file/crate choices) or implementation begin, and everything downstream is checked against this spec rather than re-litigated.

Why it's a distinct pattern, not just “detailed Planning”:

a plan describes how; a spec describes what and why, independent of implementation approach. The same spec could be satisfied by completely different plans — that separation is the whole point. This is literally what spec.md is in your SDD setup: sdd:feature drafts it, leaves explicit <!-- to fill in --> gaps for things it shouldn't guess, and sdd:plan refuses to run until that spec's status is Approved.

4. Scaffolding Pattern

USER

give an example of Scaffolding pattern

CLAUDE

Scaffolding sits one level closer to actual code than the previous three patterns. Planning decides the approach, a Spec decides the requirements — Scaffolding is about giving the model a skeleton of the shape of the code (signatures, structure, empty bodies) and asking it to fill that shape in, rather than generating a file from a blank page.

Without Scaffolding (blank-page generation):

“Write the regex-matching logic for TextFinder1's core crate.”

The model has to invent the function names, the struct layout, the error type, and the module boundaries all at once — and its choices may not match the conventions already used elsewhere in RustCmdLine / RustDirNav.

With Scaffolding:

“Here's the skeleton for the new matcher module in the core crate. Fill in the function bodies only — don't change any signatures, don't add new public items, don't reorganize anything:”

```
pub enum MatchMode {
    Substring(String),
    Regex(Regex),
}

pub struct MatchResult {
    pub line_number: usize,
    pub line_text: String,
}

impl MatchMode {
    pub fn from_args(pattern: &str, use_regex: bool) -> Result<Self, MatchError> {
        todo!()
    }

    pub fn matches(&self, line: &str) -> bool {
        todo!()
    }
}

pub fn find_matches(mode: &MatchMode, file_path: &Path) -> Result<Vec<MatchResult>,
```

```
MatchError> {  
  todo!()  
}
```

The model then only writes the bodies — constructing the regex in `from_args`, doing the actual substring/regex check in `matches`, opening the file and iterating lines in `find_matches` — while the public interface, error type, and module boundary are already fixed by you, not improvised by the model.

Why this is a distinct pattern from Planning/Spec:

a spec says what it should do; a plan says what approach to take; scaffolding goes further and fixes the literal interface shape ahead of time, so the model's freedom is constrained to implementation detail only. This is especially useful when you already know how a piece should fit into the rest of a codebase — like keeping `MatchMode` consistent with how `RustCmdLine` expects to call into the core crate — and don't want the model re-deciding that boundary on its own.

It also composes with the earlier patterns: a typical `sdd:plan` → `sdd:implement` cycle in your SDD setup could itself emit scaffolding (signatures and struct stubs) as part of `plan.md`, with `sdd:implement` then doing nothing but filling in bodies — tightening the contract even further than “follow the plan” alone would.